08. Scheduling: The Multi-Level Feedback Queue

이전까지의 문제들

FIFO Algorithm

- 간단 but convoy effect라는 큰 단점
- CPU와 device utilization이 안 좋아질 수 있음

SJF & STCF Algorithm

- Average Turnaround와 waiting time은 good
- 근데 job length를 어떻게 아느냐

RR Algorithm

- Response time 측면에서 우수
- Turnaround Time은 좋지 X

해결책

1) Turnaround Time : Shorter Job을 먼저 수행

2) Response Time : Round Robin 기법을 사용해 Process들을 최대한 동시에 수행

MLFQ

- Multi-Level Feedback Queue
- 과거로부터 미래를 예측하는 스케줄러
- 목적
  - **turnaround time 최적화** → Run shorter jobs first
  - **response time 최소화** without *a priori knowledge of job length*.

Basic Rules

- 여러 개의 다른 **queues**
  - 각 queue마다 우선순위 다름
- A job that is ready to run is on a single queue.
  - 높은 우선순위 큐에 있는 job을 골라라
  - 거기 안에서 RR 돌려라

Rule 1

If Priority(A) > Priority(B), A runs (B doesn't).

Rule 2

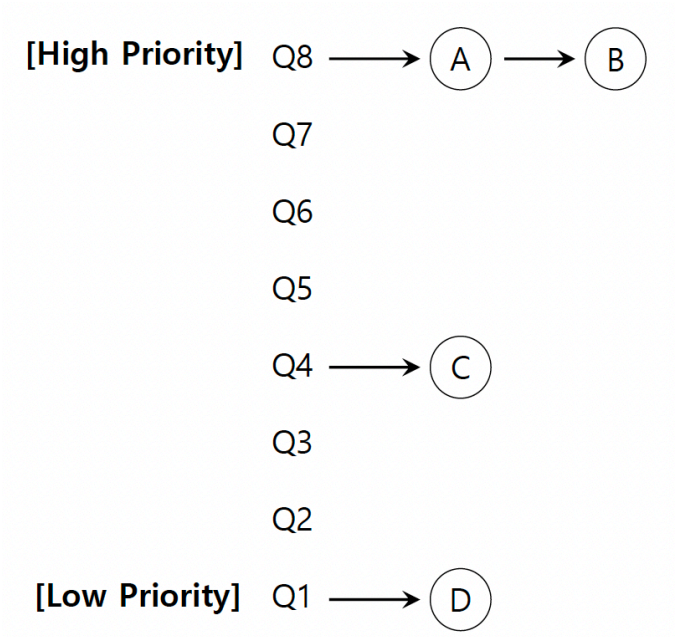
If Priority(A) = Priority(B), A & B run in RR.

우선순위는 'Observed Behavior'를 토대로 결정

I/O Operation을 위해 I/O를 자주 기다리고, CPU를 반복적으로 포기하며 적게 사용하는 Job들 → 우선순위 유지
CPU를 많이 사용하는 Job들 → 우선순위 낮춰

높이는 경우는 없다

Example



How to Change Priority

Rule 3

Job이 처음에 System이 도착하면, 가장 높은 Priority의 Queue에 위치시켜라

Rule 4a

만약 Job이 Time Quantam을 모두 소모한다면, 낮은 Priority의 Queue로 내려라

Rule 4b

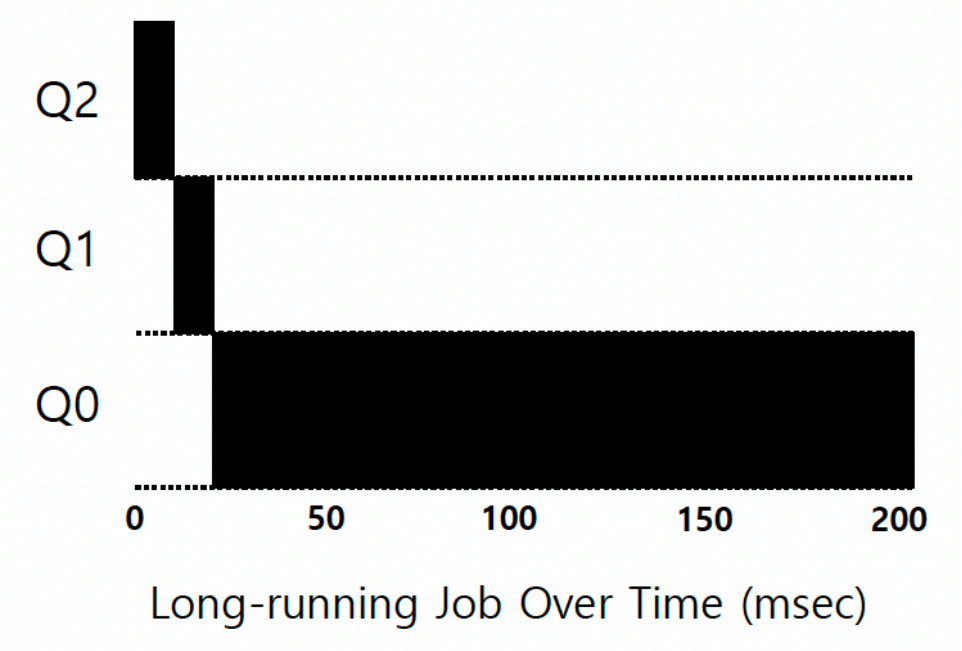
만약 Job이 Time Quantam을 모두 소모하기 전에 CPU Control을 놓는다면, 동일한 Priority Queue에 유지해라

In this manner, MLFQ approximates SJF

- CPU 길게 쓰는 job을 계속 낮춰서 우선순위가 낮아지기 때문 → B보다는 A를 먼저 선택하게 됨

Example 1: A Single Long-Running Job

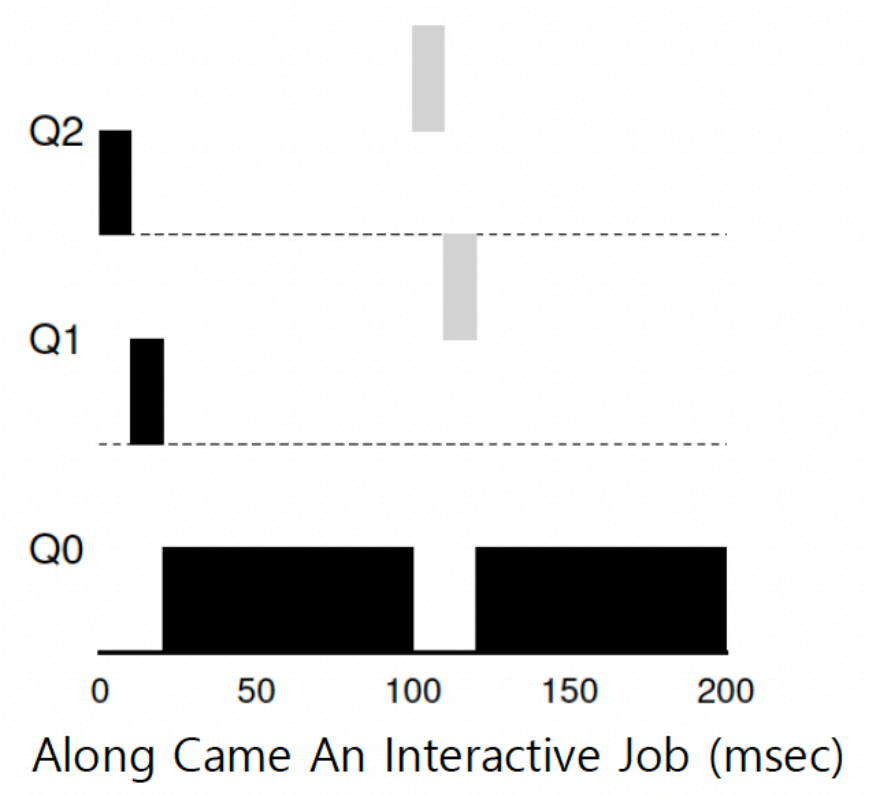
- A three-queue scheduler with time slice 10ms



- 한 간격을 10ms라고 보면 됨
- 이 긴 job은 10ms 안에서 계속 수행이 안되서 우선순위가 내려가게 되는 것

Example 2: Along Came a Short Job

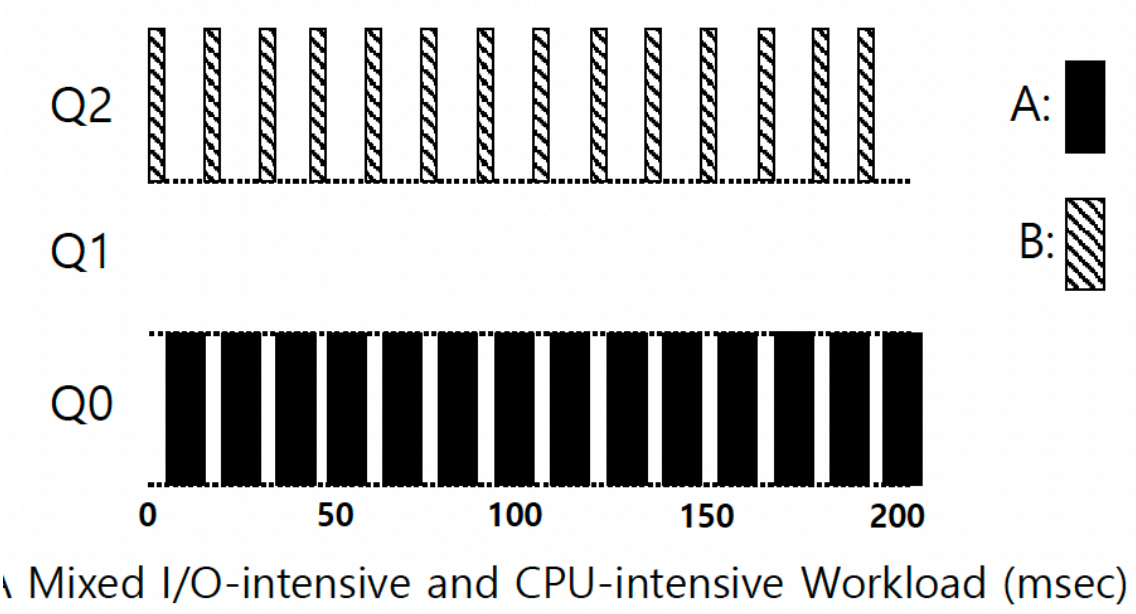
- **Job A**: A long-running CPU-intensive job (CPU 오래 씀)
- **Job B**: A short-running interactive job (20ms runtime) (CPU 짧게 씀)
- Job A가 먼저 돌다가 Job B가 T=100일 때 도착
  - 검정이 A, 회색이 B
  - B도 10ms 안에 안 끝나서 강등이 되는 것



문제: A의 Starvation

Example 3: What About I/O?

- **Job A**: A long-running CPU-intensive job
- **Job B**: An interactive job that need the CPU only for 1ms before performing an I/O



The MLFQ approach keeps an interactive job at the highest priority

Problems with the BASIC MLFQ

Starvation

- 만약 short-ruuning하는 interactive jobs이 시스템에 너무 많으면 Lon-running jobs은 CPU time을 제대로 못 받을 수 있다.

Game the scheduler

- time slice의 99%만 쓰고 I/O operation 수행
- 우선순위 유지 가능해서 CPU time 독식 가능

A program may change its behavior over time.

- CPU Bound Process와 I/O Bound Process의 성질을 모두 가지는 Process
  - 처음엔 CPU를 오래 소모하다가, 후반에 I/O를 많이 사용
  - 이러면 우선순위가 이미 낮아있는 상태라 control 잡기 어려움

> Priority Boosting이 필요하다!!

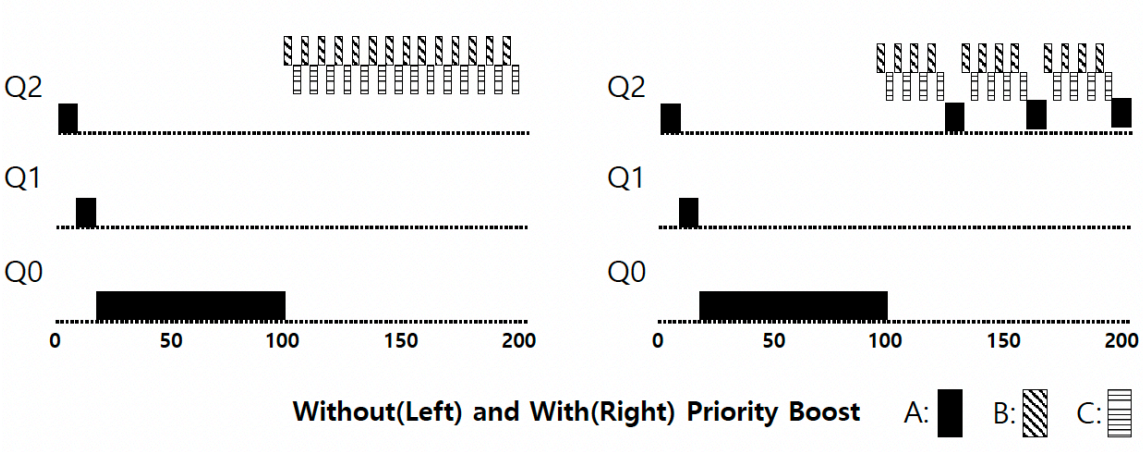
The Priority Boost

Rule 5

특정 Period S가 지나면, System의 모든 Job을 최우선순위(Topmost) Queue에 옮겨라

Example

- A long-running job(A) with two short-running interactive job(B, C)

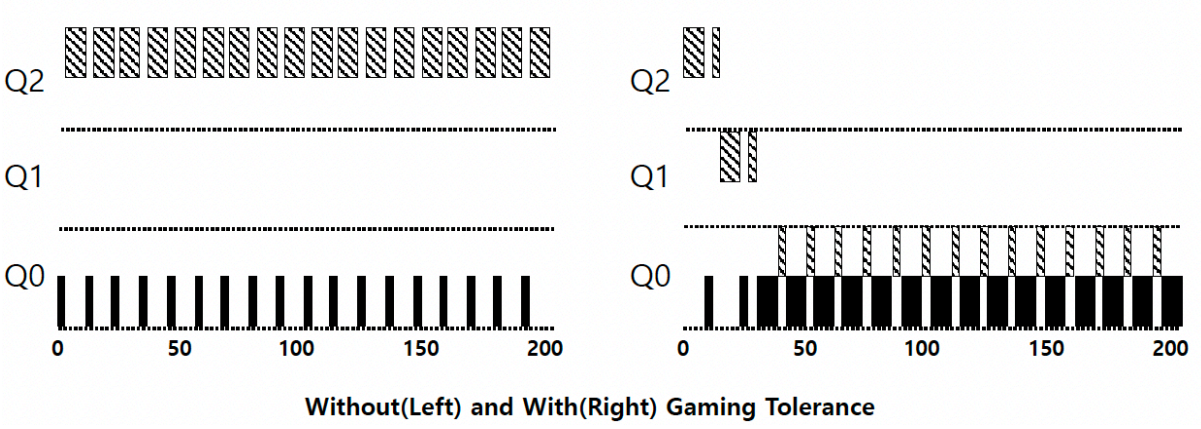


Better Accounting

- How to prevent gaming of our scheduler? - time slice 99%만 쓰는 애 방어

Rewrite Rule 4

- Rewrite Rules 4a and 4b
- 어떤 Job이 CPU Control을 놓는 횟수와는 관계없이 특정 Level의 Queue에서 '일정 시간(Time Allotment)' 이상 머무르면 Priority를 낮춰주자
  - 특정 Level에서 머무를 수 있는 시간의 Maximum

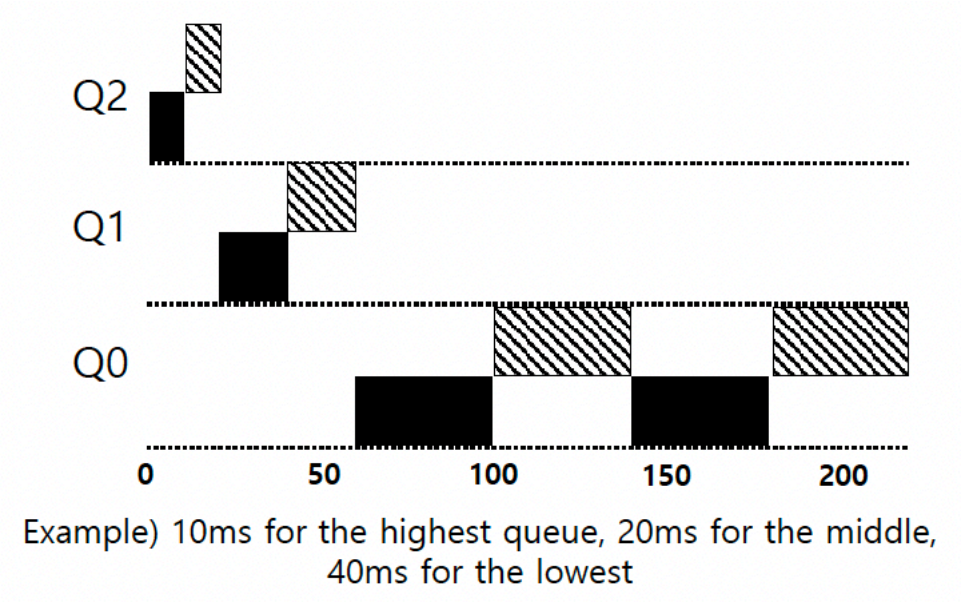


Tuning MLFQ And Other Issues

Lower Priority, Longer Quanta

- The high-priority queues →Short time slices
  - E.g., 10 or fewer milliseconds
- The Low-priority queue →Longer time slices

- E.g., 100 milliseconds



Example) 10ms for the highest queue, 20ms for the middle, 40ms for the lowest

The Solaris MLFQ implementation

For the Time-Sharing scheduling class (TS)

- 60 Queues
- Slowly increasing time-slice length
  - The highest priority: 20msec
  - The lowest priority: A few hundred milliseconds
- Priorities boosted around every 1 second or so.

MLFQ: Summary

Refined Set of MLFQ rules

Rule 1

If Priority(A) > Priority(B), A runs (B doesn't).

Rule 2

If Priority(A) = Priority(B), A & B run in RR.

Rule 3

When a job enters the system, it is placed at the highest priority.

Rule 4

Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced(i.e., it moves down on queue).

Rule 5

After some time period S, move all the jobs in the system to the topmost queue.

Beauty of MLFQ

It does not require prior knowledge on the CPU usage of a process.